Implementation of Aika, a Distributed Edge System

August Solbakken Hindenes
ahi025@uit.no

Karoline Wilsgård Benjaminsen
kbe173@uit.no

Stine Fjeldsrud Karlsen
ska161@uit.no

I. Introduction

This report presents the design and implementation of a distributed task scheduling system inspired by Áika. The objective of the assignment is to build a system which can support scalable and fault tolerant processing of large datasets across a cluster of unreliable nodes, while also ensuring strong consistency and correct task execution. The task is to label and organize 1.2 million images in a tree structure, distributing the workload.

Áika is designed to execute distributed AI applications on edge systems in unreliable and resource constrained environments [1]. The overall system architecture follows Áika's hierarchal design consisting of *cluster controllers*, *local controllers* and *worker agents*.

To guarantee consistency, each task shall be processed only once, without duplication, loss, or inconsistency even with failures. That is, each image should only be labeled and organized once. To ensure this, the system is implemented as a replicated state machine with a consensus protocol based on Raft, a leader-based consensus algorithm [2].

The system is then evaluated through experiments measuring throughput, scalability, and fault tolerance under varying configurations and failure scenarios. The results show that the system achieves reliable task completion and stable performance.

II. Background

Distributed systems operating in unreliable environments must ensure both fault tolerance and consistent system state despite failures. Two key concepts for such systems are hierarchical task execution and consensus-based state replication.

Áika proposes a distributed edge architecture designed for unreliable environments with limited connectivity, high latency, and strict privacy constraints. In Áika, computation is organized in a hierarchical controller-agent architecture. Worker agents perform the actual data processing, the local controllers monitor agents on the same node, and the cluster controllers are responsible for ensuring that the physical nodes are running, initiating replication in case of failure [1]. This separation enables the system to remain operational even if a component fails through replication and decentralized coordination.

Consensus protocols are used to ensure consistent system states in distributed environments. Raft is a consensus protocol designed with leader election, log replication, and safety guarantees. In Raft, a single leader is responsible for accepting client requests and replicating them to followers, such that all nodes eventually execute the same sequence of operations. Once an operation is replicated across a majority of nodes, it is considered committed and can be safely applied to the system's state machine [2]. This guarantees strong consistency even in the presence of failures.

Together, these concepts enable the design of distributed scheduling systems where tasks can be coordinated reliably across multiple nodes. A hierarchical execution architecture ensures robustness, and consensus-based replication ensures consistency.

The implementation presented in this report build on these ideas by implementing a scheduling system based on Raft and Áika, that distributes task batches across worker agents while ensuring that tasks are neither duplicated, invalid or lost.

III. Design

The system design combines aspects from Áika and Raft, along with system-specific scheduling mechanisms.

A. Áika-Inspired Hierarchical Architecture

The overall system follows the three-tier hierarchical architecture described in the Áika paper [1]. Computation is organized into three roles: cluster controllers, local controllers, and worker agents.

The **cluster controllers** form a replicated coordination layer. This layer is responsible for global task scheduling and coordination across the entire cluster, maintaining a consistent view of task assignments. It is also responsible for fault tolerance, managing recovery at the system level if a node crashes. To avoid complete system failure in the event of a cluster controller crash, the controller is duplicated and controllers are connected in a chain, where one is the leader and the others are replicas [1].

The **local controllers** operates on physical nodes, and manage and monitor the worker agents on their nodes. It checks that each agent is running, and restarts an agent in the event of a failure. If a local controller has no worker agents, it is a replica used in recovery of node failures [1].

Finally, the **worker agents** perform the actual computation by processing images with feature extraction and labeling tasks [1].

This hierarchical architecture allows failures to be handled at different levels of the system. An agent failure can be resolved locally on a node without affecting the rest of the cluster. If a node fails, it is handled by the cluster controllers.

This distribution of responsibility ensures robustness and improves scalability by avoiding centralized bottlenecks.

B. Raft-Based Consensus and State Management

Global consistency is achieved through a replicated state machine coordinated using the Raft consensus protocol, as described in the Raft paper [2]. All cluster controller maintain a replicated log of commands representing task and node state. The system uses Raft's leader model were a single leader is responsible for accepting all state updates and coordinating replication to the controllers [2].

Leader election follows Raft's randomized timeout mechanism. Each controller initially operates as a follower and expects periodic heartbeats from the current leader. If these heartbeats are not received within a specified timeout interval, the replicas enter a candidate state and initiates election by requesting votes from other controllers. A controller that receives the majority of the votes from other replicas becomes the new leader [2].

The established leader becomes responsible for log replication. Each node and task state change is appended to the leader's log as a new entry and then replicated to the other controllers using AppendEntries messages. The followers verify consistency by checking that the preceding log entry matches in both index and term. If any inconsistencies are detected, the conflicting entries are removed before new entries are appended [2]. By doing this, all logs converge into the same sequence over time.

A log entry is considered committed when it has been replicated on a majority of the followers. Only committed entries are applied to the state machine, which guarantees that all controllers execute the same sequence of operations [2].

To support recovery, each follower maintains a persistent state, including the current term, voter information, and log entries [2]. This allows nodes to recover from crashed and rejoin the cluster.

This design, including leader election, log matching, state machine safety and persistent state, ensures that the system behaves like a single, consistent entity despite failures.

C. System-Specific Design

a) *Task Scheduling:* On top of the Raft consensus, the system implements task scheduling as a deterministic state machine. All task related operations are replicated through the Raft log, ensuring that every controller maintains an identical view of task and node states.

Tasks are organized into batches and progress through a defined life cycle. A task begins in a pending state, before it is assigned to a worker. The task then transitions to either completed or back to pending. A task returns to pending if it exceeds its time-to-live (TTL) period and is considered expired. These task transitions are driven exclusively by commands through the replicated Raft log, which ensures that task ownership and completion are consistent on a global scale. By doing this, the system guarantees that tasks are completed without duplication, loss or invalid results, even in the presence of failures.

b) *Batching:* By grouping tasks into batches before being committed to the system, the overhead of the consensus is reduced. Each batch corresponds to a single log entry, reducing the number of consensus operations compared to per-task (image) scheduling.

c) *Pull-Based Scheduling:* Task scheduling follows a pull-based model. Worker agents request tasks when they become idle, rather than being assigned work. This results in implicit load balancing, as faster nodes or nodes with more available resources will naturally process more tasks. The approach avoids the need for a centralized scheduler with knowledge of system state and capacity.

D. Failure Handling and Recovery

Failure handling combines mechanisms inspired by Áika with safety guarantees provided by Raft. The system assumes a fail-stop failure model where nodes either operate correctly or crash.

At the consensus level, Raft ensures that the system can continue to operate as long as the majority of cluster controllers are available. The leader election allows the system to recover automatically if a cluster controller fails with minimal disruption.

At the node level, local controllers detect failures of worker agents, remove them and restart them immediately. If the failed agent's assigned task batch expires and is returned to pending for reassignment.

In the event of a node failure, where a local controller stops working, cluster controllers coordinate recovery by reallocating work to a standby local controller replica. The failed controller is restarted as a new standby replica.

IV. Implementation

The implementation of the system follows the architectural principles outlined in the design section. However, several practical considerations, optimizations, and constraints were necessary for stable execution on the cluster.

A. Raft Protocol Implementation

The Raft protocol is implemented closely following the Raft paper [2], particularly for leader election, log replication, commits, and persistent state. Leader election uses randomized timeouts in the range of 1000-3000 ms, combined with periodic heartbeats at 200 ms intervals. This ensures fast detection of failures while avoiding excessive election conflicts.

A challenge occurs during controller restart. When a cluster controller restarts with its persisted term, it may trigger a new election before receiving a heartbeat from the current leader. According to Raft's rules, this forces the leader to step down, even if the restarting controller cannot win the election, causing a short disruption in the system. Resetting the persisted term was considered as a workaround, but this would violate Raft's election safety guarantee by allowing a node to vote multiple times during the same term. Instead, the implementation preserves consistent state and mitigates instability by resetting the election timer whenever a node steps down due to a higher-term request. This reduces repeated

election, although each restart still causes a brief re-election delay.

Log replication follows the standards AppendEntries mechanism with consistency checks. To improve performance during conflict, the implementation includes fast backtracking, where the replicas return a conflict index that allows the leader to skip directly to the correct log position instead of retrying incrementally. This reduces the number of failed replication attempts in practice.

The log is stored as an append-only NDJSON file with two write paths. The incremental appends are used for normal operation, while full rewrites are only triggered during conflict resolution or recovery. This ensures that the common case, leader proposal and conflict free appending, never rewrites the entire log, minimizing disk overhead while maintaining correctness.

If a node crashes mid-write, the log may contain an invalid, truncated entry. During recovery, the log is read until the first invalid entry, after which the remaining data is discarded and the file is rewritten to remove corruption. This prevents repeated failures due to persistent invalid entries. Additionally, newly elected leaders append a no-op entry to ensure correct commits, and this entry is immediately persisted to avoid gaps in the log if a crash occurs shortly after election.

Several optimizations were implemented to improve performance. Replication to each controller is handled using independent asynchronous tasks, ensuring slow or failed nodes do not stall progress. The log proposals are also pipelined, allowing up to 16 task batches to be proposed concurrently instead of waiting for each commit, keeping the Raft pipeline saturated during the ingestion phase.

Certain features from the Raft paper were intentionally not implemented. Log compaction through snapshots and dynamic membership were omitted to reduce complexity. Without snapshots the log grows unbounded for the lifetime of the cluster. For 1.2 M images at batch size 50, this is approximately 15 k log entries (10 MB), which is negligible. Without dynamic membership, the system operates with a static set of cluster controllers. Pre-vote was also not implemented, which contributes to the restart-induced election disruptions described earlier, but this trade-off was considered acceptable given the added complexity and the assignment constraints.

B. State Machine and Task Distribution

Task scheduling is implemented as a replicated state machine. All task and node operations are encoded as commands in the log, including adding task batches, assigning tasks, completing tasks, expiring tasks and registering a node in the cluster. All commands are idempotent in the state machine, meaning that applying the same command twice produces the same result as applying it once. So if a batch is completed twice, only the first completion is recorded. This ensures correctness when commands are retried, which can occur during leader changes or log inconsistencies.

Task ingestion is performed by the leader, which scans the dataset, chunks filenames into batches, and proposes batch creation commands. If a leader change occurs, ingestion is resumed by re-scanning and skipping already present in the replicated state machine, ensuring no duplicate tasks. Each task batch has a time-to-live (TTL). A periodic reaper process on the leader scans for expired tasks and proposed expiration commands, returning the task batches to the pending state for reassignment.

C. Failure Handling

Controller or agent failures are implemented as described in the design section. An external watchdog is responsible for process-level recovery.

Cluster controller failure is handled by Raft's leader election. In addition, a watchdog monitors controller processes using periodic system checks. If a controller process is no longer running, the watchdog can detect and restart it within minimum 5 seconds. However, a 10 second delay was added from detection to the restart process starts so that the effects of controller crashes can be observed. The cluster controller rejoins as a replica using its persistent state and catches up via log replication.

Local controller failures are detected by the current cluster controller leader using heartbeat monitoring with a 3-second interval. A two stage detection process is used in practice. One missed heartbeat marks the node as suspected, while multiple missed heartbeats confirm the node as down. Upon confirmation, the cluster controller activates a standby replica by passing the failed node's agent count, script path and image base path. The watchdog then restarts the failed local controller after a delay to allow the replacement to take over first. The restarted controller is readded as a replica. In addition, the system maintains a target number of active local controllers, and can promote or demote nodes dynamically during runtime.

Worker agents are monitored locally by the local controller using process-level checks (try_wait() call). Failed agents are restarted. Any work assigned to a failed agent is recovered through the TTL mechanism, where expired tasks are detected and reinserted in the pending queue.

D. Performance Optimizations

The implementation includes several optimizations to the workload. Since the system is primarily I/O-bound due to a shared file system storage, each worker agent is restricted to a single computation thread. This allows a larger number of agents to run concurrently. System responsiveness is improved by reducing heartbeat intervals and failure detection timeouts, significantly reducing recovery latency. Additionally, pending tasks are stored in a deque structure, where stale entries (already assigned or completed) are skipped on each dequeue.

V. Experiments

The system was evaluated through scalability, configuration, fault tolerance, and full-run experiments.

Firstly, to demonstrate the correctness of the system, a full run was performed. The configuration was 50 local controllers, 8 agents per node, batch size 50 and TTL at 180 seconds.

Scalability experiments on 100,000 images measure how parameters such as number of local controllers, agents, and batch size affect throughput. When a specific parameter was not being tested, they were set to 20 local controllers, 8 number of agents and batch size of 100 images.

Optimal configuration experiments on 200,000 images were performed to identify which parameter combination yields the highest performance.

Fault tolerance is tested by initiating failures in cluster controllers and local controllers, observing the recovery and impact on throughput.

Lastly, an additional experiment with the full dataset of 1.28 million images and the optimal configuration was run to test if limitations were caused by NFS I/O rather than the scheduling system.

VI. Results

As seen in figure 1, the system successfully processes the full dataset of 1.28 million images in 52 minutes, achieving an average throughput of 405 images per second. Throughput has an initial decline, but stabilizes after, confirming that the system maintains correctness despite varying performance conditions.

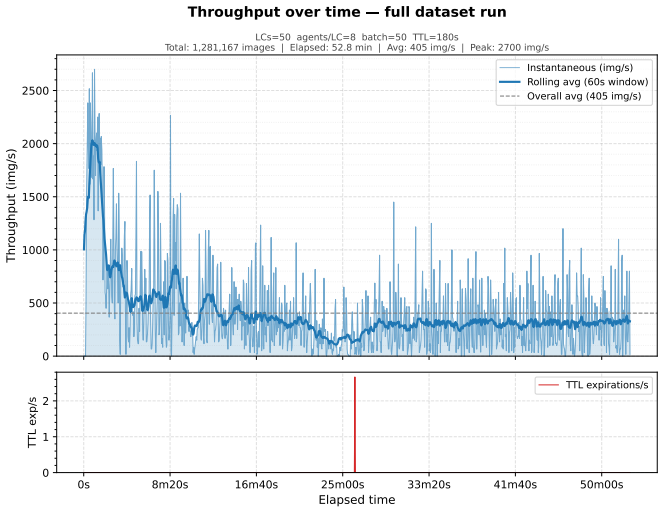

Fig. 1. Throughput over time during a full dataset run.

As seen in figure 2, throughput increases almost linearly with the number of local controllers. It increases from approximately 264 images per second at 5 nodes to 1639 images per second at 50 nodes. We can also see that as throughput increases, completion time decreases accordingly.

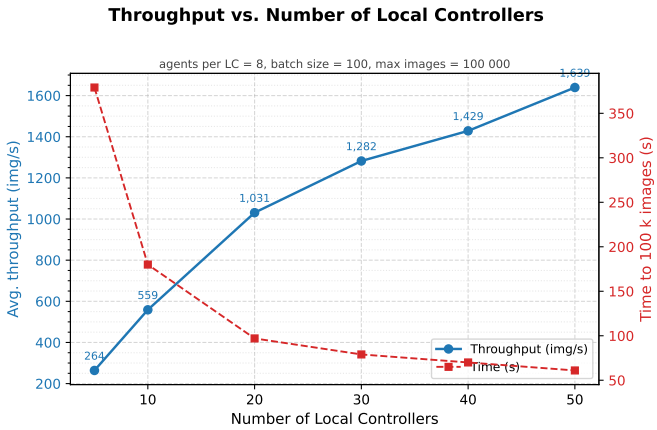

Fig. 2. Average throughput and completion time with varying number of local controllers.

When increasing the number of agents per node, as seen in figure 3, the throughput improves up to 8 agents at roughly 1000 images per second. After this, performance degrades, probably due to resource contention. For completion time, we see the opposite graph. Completion time decreases up to 8 agents, after which it starts to increase again.

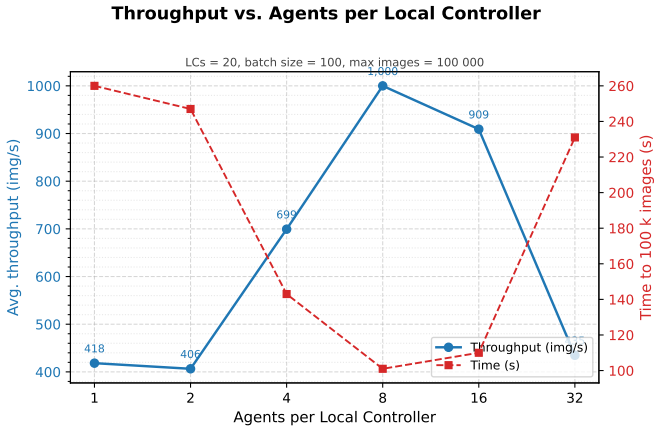

Fig. 3. Average throughput and completion time with varying number of agents per local controller.

Similarly, in figure 4, we see that when increasing batch size, throughput increases up to batch sizes 100-200, then drops for larger batches. Here, the completion time also show the opposite pattern: decreasing up to 100 to 200 images, then increasing.

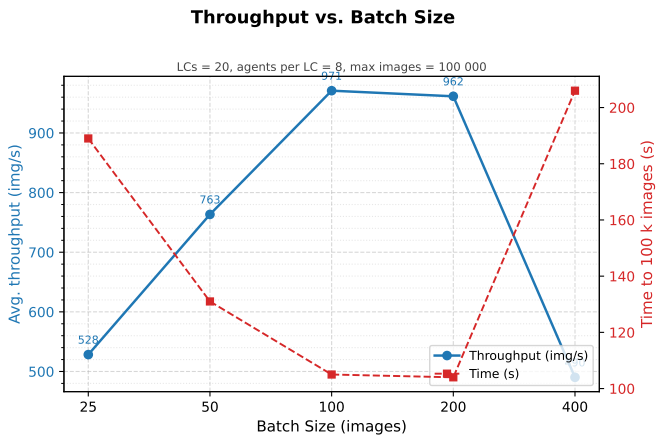

Fig. 4. Average throughput and completion time with varying batch sizes.

Figure 5 shows the ranking of different combinations of parameter values, changing number of local controllers, number of agents and batch size. The ranking is based on the average throughput and completion time. Based on this, the best performing configuration is 50 local controllers with 8 agents per node and batch size 200. This configurating achieves throughput of 2041 images per second and completing 200,000 images in 98 seconds. Having a larger batch size of 200, consistently outperforms having a batch size of 100 at the same scale.

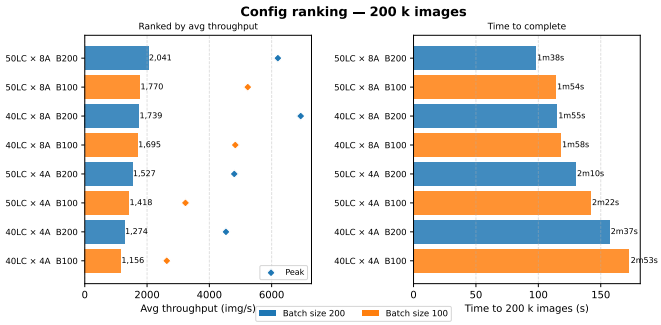

Fig. 5. Ranking of different configurations based on their average throughput and completion time

Figure 6 shows the result of 7 cluster controller failures and 7 cluster restarts. Cluster controller failures cause a short throughput drop corresponding to leader re-election of 1-2 seconds. After this the system quickly recovers and maintains an average throughput of 995 images per seconds. As seen in the bottom graph we get only 39 TTL expirations in total, meaning almost no work was lost.

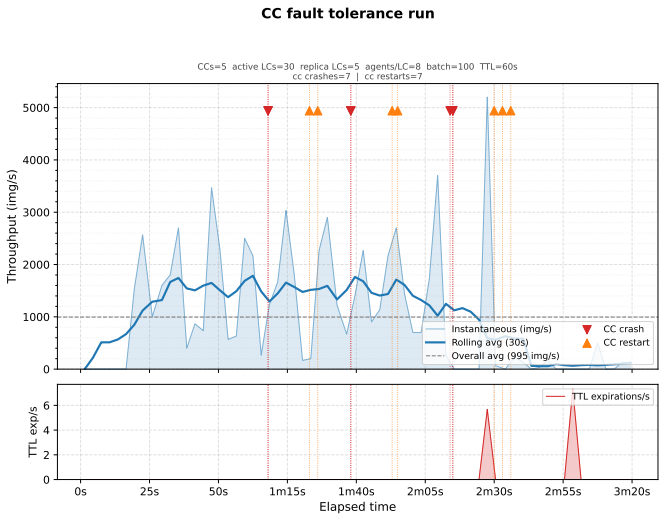

Fig. 6. Cluster controller failures.

As seen in figure 7, local controller failures with 23 crashes and 23 restarts result in longer recovery periods of 15 seconds from watchdog delay and higher task reassignments. This is reflected in increased TTL expirations, lower average throughput of 885 images per second and bigger dips in instantaneous throughput. Despite this, the system completes all tasks successfully.

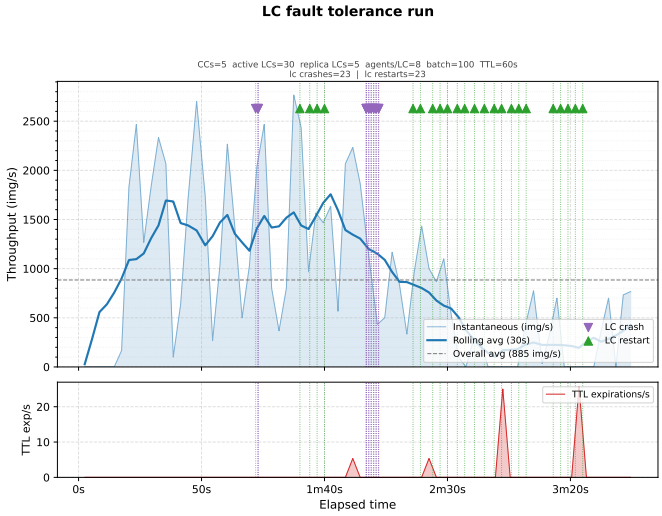

Fig. 7. Local controller failures.

Figure 8 shows a full run with the optimal configuration of 50 local controllers, 8 agents and batch size of 200 images. The system reaches throughput of 1326 images per second for the first 7 minutes/640,000 images, then stalls completely. We can also see that the TTL rate rises after stalling.

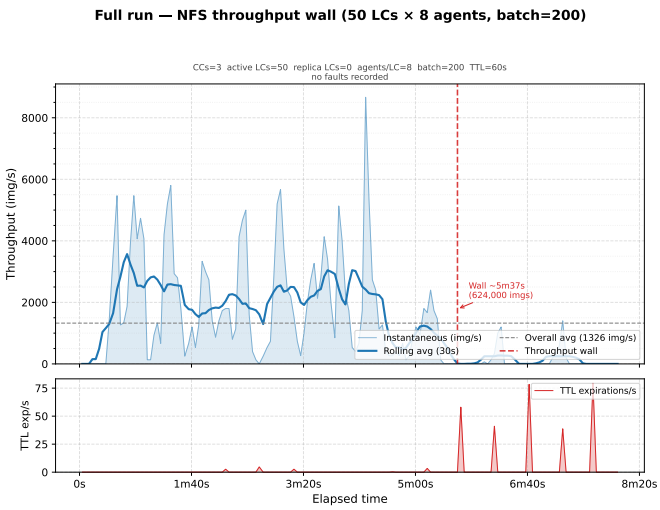

Fig. 8. Full run with optimal configuration showing a throughput wall.

VII. Discussion

The results demonstrate that the system achieves scalable, consistent, and fault tolerant task execution, while also revealing practical limitations related to resource contention and external I/O constraints.

A. Scalability Evaluation

The system demonstrates neal-linear scalability with respect to the number of local controllers, as shown in figure 2. Increasing the number of nodes results in increased throughput and decreased completion time, indicating that the system effectively distributes work across the cluster, making it scalable. However, scalability is not uniform across all parameters. Increasing the number of agents per node and the batch size only improves performance up to a certain point, after which throughput decreases. These results show that while the system scales horizontally, performance is sensitive to resource usage within each node.

B. Throughput Evaluation

The system achieves high throughput under optimal configurations, reaching over 2000 images per second in the best case. This proves that batching images and pipelined Raft proposals reduce coordination overhead.

The full dataset run demonstrates that the system can sustain long-running workloads and maintain stable throughput over time. However, the experiment with the optimal configuration on the full dataset shows a sharp performance drop where the system grinds to a halt. Initially, we suspected that this degradation might stem from Raft log consensus overhead, as the replicated log grows with each ingested record and could theoretically impose an O(log N) slowdown on commit latency. However, Figure 1 shows that throughput stabilizes after the drop, and in the latter half of the run we observe very stable throughput. This is inconsistent with a log-replication bottleneck, which would produce a progressively worsening decline. Furthermore, the sharp decrease consistently occurred at approximately the same point in the workload, and this point shifted when we changed the order in which the images were ingested. This suggests that the cause is tied to a specific region of the filesystem rather than cumulative system state. The most likely explanation is that the NFS server's directory cache eventually saturates due to the large number of files. Once we exceed what can be cached, read latency spikes to the point where we are no longer able to read all files in a batch within the task TTL, explaining the wall observed in Figure 8.

C. Fault Tolerance

The system shows robustness to both cluster controller and local controller failures. Cluster controller failures result in short throughput drops corresponding to Raft re-election, but the system recovers with minimal task loss. This confirms that Raft-based consensus provides fault tolerance and system consistency.

Local controller failures have a more noticeable impact on performance, as they result in longer recovery times and reassignment of tasks. This is reflected in the increased TTL expirations, temporary throughput degradation and lower throughput average. Despite this, the system consistently recovers and completes all tasks, showing that the combination of watchdog-based recovery and TTL-based reassignment of tasks ensures eventual completion.

D. Consistency Guarantee

The system satisfies the requirement that each task is completed exactly once. This is achieved through the replicated state machine and idempotent task operations. The full dataset run confirms that all images are processed without duplication or loss.

E. Limitations

The system also exhibits some limitations. The use of a single Raft leader introduces a potential throughput bottleneck, as all scheduling decisions must be serialized through the leader's replicated log. As the number of workers increases, the rate of task assignment and completion proposals may limit overall system performance.

Another challenge is the trade-off between responsiveness and stability in the Raft implementation. While aggressive timeout tuning improves recovery time, it can also increase the chance of unnecessary elections during restart. Additionally, not implementing features such as pre-vote simplifies the implementation but introduces minor inefficiencies and limitations.

However, the primary limitation observed in the experiments is the dependence on shared NFS storage. The sharp throughput drop in figure 8 indicates that the system becomes I/O bound when operating on large datasets, particularly when accessing files located in slower regions of the file system. This limitation is not caused by the scheduling system but significantly impacts overall performance.

VIII. Conclusion

This report presented a distributed task scheduling system based on Áika, implemented using Raft consensus. Overall,

the system meets the requirements of the assignment by achieving scalable task execution, strong consistency and robust fault tolerance. Experimental results show that the system performs well under varying configurations and failure scenarios. Performance is primarily limited by external factors such as I/O bandwidth rather than the system itself.

IX. Appendix

A. AI Declaration

This report was partially assisted by AI-based tools. ChatGPT-5.3 was used for creating report structure, brainstorming result explanations, and minor language refinement.

The code project was developed with assistance from Claude Code (Anthropic), using a combination of Claude Sonnet 4.6 and Claude Opus 4.6 in both ask and agent mode. All agentic actions were thoroughly reviewed by a human before being accepted.

Specific uses:

- Watchdog dashboard: The entire watchdog/deploy dashboard and the watchdog functionality was generated by an AI agent with human guidance. This was done to save time as it is not a core part of the assignment.
- Code comments: Agent mode was used extensively to write descriptive comments throughout the codebase.
- Concurrency assistance: Agent mode helped with thread-safe code patterns and troubleshooting related bugs.
- Raft tests: Agent mode was used to set up the Raft consensus test suite.
- Documentation: README was partially generated and updated with AI assistance (API, Flags, Architecture sections).

All final decisions, code, and written content were reviewed and verified by the authors, who take full responsibility for the work.

References

[1] J. A. Alslie *et al.*, "Áika: A Distributed Edge System for AI Inference," *Big Data and Cognitive Computing*, vol. 6, no. 2, p. 68, 2022, doi: 10.3390/bdcc6020068.

[2] D. Ongaro and J. Ousterhout, "In Search of an Understandable Consensus Algorithm (Extended Version)," 2014.